Ques. What is Lambda/Anonymous Function?

- A lambda function is a small anonymous function(**anonymous function** is a function that is defined without a name).
- While normal functions are defined using the **def** keyword in Python, anonymous functions are defined using the **lambda** keyword.
- **Note** The anonymous function does not have a **return keyword**, the anonymous function will **automatically return** the result of the expression in the function once it is executed.

Syntex

```
lambda arguments: expression

# For Example
lambda x: x * 2

# here
lambda → keyword
x → argument
: → separates arguments and expression
x * 2 → expression/result
```

```
# Normal Function
def add(a,b):
  print(a+b)
add(5,10) # Output:- 15

# Using Lambda function
x = lambda a: a + 10
print(x(5)) # Output:- 15
```

Lambda with One Parameter

```
square = lambda x: x * x

print(square(5)) # Output:- 25

# Equivalent:
def square(x):
    return x * x
```

Lambda with Multiple Parameters

```
add = lambda a, b: a + b
print(add(10, 20))  # Output:- 30

# Example 2
multiply = lambda a, b, c: a * b * c
print(multiply(2, 3, 4))  # Output:- 24
```

Lambda Without Arguments

```
message = lambda: "Hello Python"

print(message())  # Output:- Hello Python
```

Lambda with Default Arguments

```
greet = lambda name="Mohit": f"Hello {name}"

print(greet())        # Output:- Mohit
print(greet("Rahul")) # Output:- Rahul
```

Lambda with if-else

```
check = lambda x: "Even" if x % 2 == 0 else "Odd"

print(check(10))  # Output:- Even
print(check(7)) # Output:- Odd
```

Lambda with max()

```
numbers = [10, 25, 5, 40, 15]

result = max(numbers, key=lambda x: x)
print(result) # Output:-40

# Example2
students = [
    ("Mohit", 85),
    ("Rahul", 92),
    ("Amit", 78)
]

result = max(students, key=lambda x: x[1])
```

```
print(result) # Output:- ('Rahul', 92)

# Details:- Compare students based on their marks.
```

Lambda with min()

```
students = [
    ("Mohit", 85),
    ("Rahul", 92),
    ("Amit", 78)
]

result = min(students, key=lambda x: x[1])
print(result) # Output:- ('Amit', 78)
```

Lambda with sorted()

```
students = [
    ("Mohit", 85),
    ("Rahul", 92),
    ("Amit", 78)
]

result = sorted(students, key=lambda x: x[1])

print(result) # Output:- [('Amit', 78), ('Mohit', 85), ('Rahul', 92)]

# Descending order:
result = sorted(
    students,
    key=lambda x: x[1],
    reverse=True
)

print(result) # Output:- [('Rahul', 92), ('Mohit', 85), ('Amit', 78)]
```

Lambda with map()

- map() applies a function to every element.

```
numbers = [1, 2, 3, 4, 5]

result = map(lambda x: x * 2, numbers)
print(list(result)) # Output:- [2, 4, 6, 8, 10]
```

Lambda with filter()

- filter() is used to select elements based on a condition.

```
numbers = [1, 2, 3, 4, 5, 6]

result = filter(lambda x: x % 2 == 0, numbers)

print(list(result)) # Output:- [2, 4, 6]
```

Lambda with reduce()

- reduce() is available from the functools module.

```
from functools import reduce

numbers = [1, 2, 3, 4, 5]

result = reduce(lambda x, y: x + y, numbers)

print(result) # Output:- 15

# How it work
1 + 2 = 3
3 + 3 = 6
6 + 4 = 10
10 + 5 = 15

# Example2
from functools import reduce

numbers = [1, 2, 3, 4]

result = reduce(lambda x, y: x * y, numbers)

print(result) # Output:- 24
```

Can Lambda have *args?

```
add = lambda *args: sum(args)

print(add(1, 2, 3, 4)) # Output:- 10

#
def add(*args):
    return sum(args)
```

Can Lambda have **kwargs

```
show = lambda **kwargs: kwargs

print(show(name="Mohit", age=30)) # Output:- {'name': 'Mohit', 'age': 30}
```

Lambda with *args and **kwargs

```
func = lambda *args, **kwargs: {
    "args": args,
    "kwargs": kwargs
}

print(func(10, 20, name="Mohit"))

# Output:-
{
    'args': (10, 20),
    'kwargs': {'name': 'Mohit'}
}
```

- We can use lambda function in **filter()**

```
# filter() function is used to filter a given iterable (list like object) using
another function that defines the filtering logic.
# Syntex:- filter(object, iterable)
# The object here should be a lambda function which returns a boolean value.
mylist = [2,3,4,5,6,7,8,9,10]
list_new  = list(filter(lambda x : (x%2==0), mylist))
print(list_new) # Output:- [2, 4, 6, 8, 10]
```

- We can use lambda function in **map()**

```
# map() function applies a given function to all the itmes in a list and returns
the result. Similar to filter(), simply pass the lambda function and the list (or
any iterable, like tuple) as arguments.

mylist = [2,3,4,5,6,7,8,9,10]
list_new  = list(map(lambda x : x%2, mylist))
print(list_new)

Output:- [0, 1, 0, 1, 0, 1, 0, 1, 0]
```

- You can use lambda function in **reduce()** as well

```
# reduce() function performs a repetitive operation over the pairs of the elements
in the list. Pass the lambda function and the list as arguments to the reduce()
function. For using the reduce() function, you need to import reduce from
functools librray.

from functools import reduce
list1 = [1,2,3,4,5,6,7,8,9]
sum = reduce((lambda x,y: x+y), list1)
print(sum)

Output:- 45 //i.e 1+2, 1+2+3 , 1+2+3+4 and so on.
--------------------------------------------------------------------------------
# How to use lambda function to manipulate a Dataframe
# You can also manipulate the columns of the dataframe using the lambda function.
It’s a great candidate to use inside the apply method of a dataframe. I will be
trying to add a new row in the dataframe in this section as example.

import pandas as pd
df = pd.DataFrame([[1,2,3],[4,5,6]],columns = ['First','Second','Third'])
df['Forth']= df.apply(lambda row: row['First']*row['Second']* row['Third'],
axis=1)
df

Output:-
|   | First | Second | Third | Forth |
| 0 |  1    |    2   |   3   |  6    |
| 1 |  4    |  5     |   6   |  120  |
```